

Text - to - SQL

Introduction

Text to SQL 是一種自然語言處理(NLP)技術，將使用者輸入的自然語言轉換為結構化的 SQL 查詢。透過這項技術，**使用者不需要學習複雜的 SQL 語法**，只需使用日常語言即可與資料庫進行互動，讓非技術使用者也能輕鬆查詢資料。

Example of Text to SQL:

Natural Language (Text):

- "Show me all employees from the Sales department."

SQL Query:

sql

 複製

```
SELECT * FROM employees WHERE department = 'Sales';
```

01

方法

基於規則與模板

需要專家預先編寫規則，使用語法規則來解析自然語言並生成 **SQL**

1. 資料字典 (Data Dictionary)

儲存資料庫結構資訊，幫助Text-to-SQL 系統理解表、欄位及其關聯關係

Field Name	Data Type	Description	Example
full_name	Text	Student's official full name	Bob
contact_no	Text	Student's phone/contact number	0912345678
curr_address	Text	Student's current living city	Khulna

基於規則與模板

需要專家預先編寫規則，使用語法規則來解析自然語言並生成 SQL

2. 上下文無關文法 (Context-Free Grammar, CFG)

使用遞迴規則來生成語法樹 (Parsing Tree) 解析自然語言句子，將其轉換為適合 SQL 生成的結構化表示，只有符合語法的語句才能被合法產生。

PERSONAL-INFO	→	ROLL FULL_NAME F_NAME BTCH B_DATE BLD_GRP SPOUSE_NAME NO_OF_CHILDREN FLD_OF_INTEREST JOB_TITLE COMPANY_NAME JOB_EXP
CONTACT-INFO	→	ID CONTACT_NO EMAIL_ID FAX_NO WEBSITE_INFO CURR_ADDRESS
ROLL	→	roll number roll no
FULL_NAME	→	who is name name of the student student name student's name
F_NAME	→	father father's name dad
BTCH	→	Batch
B_DATE	→	birthday birth date date of birth
BLD_GRP	→	blood group
SPOUSE_NAME	→	spouse name husband wife name of husband husband's name name of wife wife's name

“Show the student's name and contact no who lives in Khulna.”

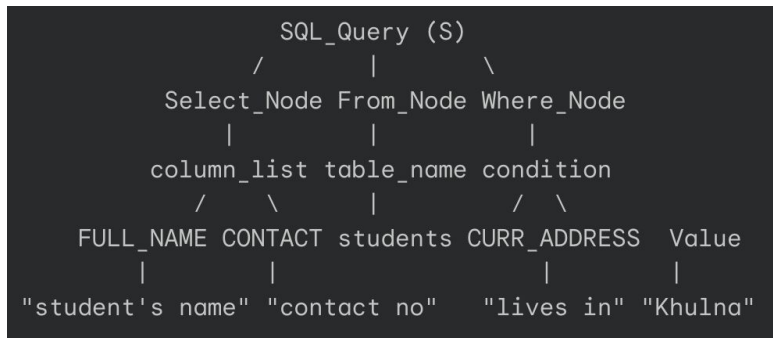


Fig. 2: Context Free Grammar for SELECT and FROM clause

基於規則與模板

需要專家預先編寫規則，使用語法規則來解析自然語言並生成 **SQL**

3. 規則庫(Rule Base)

儲存各種 SQL 生成的規則，根據查詢類型自動應用對應的SQL 結構

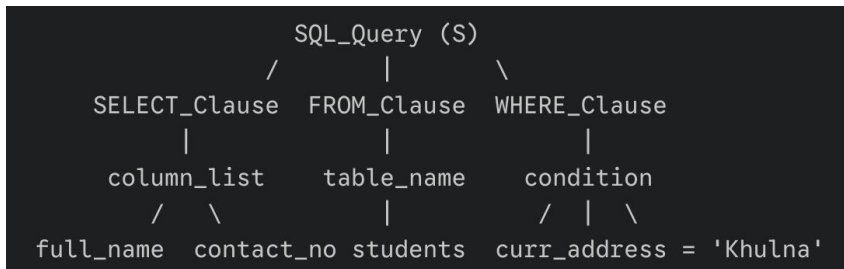
Rule ID	類型	語意模式	對應 SQL 結構
R1	選擇欄位	“show the {column_list}”	SELECT {columns}
R2	查詢表格	預設 students 表	FROM students
R3	地點條件	“lives in {curr_address}”	WHERE curr_address = '{curr_address}'

基於規則與模板

需要專家預先編寫規則，使用語法規則來解析自然語言並生成 SQL

4. 基於data dictionary 和規則庫組裝SQL句子

“Show the student's name and contact no who lives in Khulna.”



```
SELECT full_name, contact_no
FROM students
WHERE curr_address = 'Khulna';
```


基於規則與模板

優點:

- 相同的輸入問題會得到一致的查詢結果, 減少查詢生成的變異性
- 不需要大量的訓練數據、具有較強的可解釋性

缺點:

- 維護成本高(需人工添加規則)
- 難以適應不同的資料庫結構及複雜的問題查詢

基於深度學習

隨著深度神經網路的興起，Seq2Seq 和 Transformer 架構逐漸取代規則方法，能從大量標註資料中自動學習自然語言到 SQL 的映射

1. 序列到序列模型 (Sequence-to-Sequence, Seq2Seq)

使用編碼器將輸入的自然語言與資料庫結構轉換為語義表示；使用解碼器根據語義表示逐步生成 SQL 語句

2. Transformer 架構

Self-Attention 允許模型在處理時關注整個輸入句子，從而更好地理解語境；Multi-Head Attention 允許從不同角度學習語義關聯

基於深度學習

缺點:

- 需要大量標註資料集, 在新的資料庫(無標註數據)上, 模型可能無法準確生成 SQL
- 如果模型在訓練中沒有見過特定資料庫結構, 它可能會錯誤地選擇欄位或表, 生成符合語法結構, 但無法執行的SQL

employees 表格

id	1	2	3	4
name	Alice	Bob	Charlie	David
department_id	101	102	101	103
salary	50000	45000	55000	47000

departments 表格

id	101	102	103
department_name	Engineering	HR	Marketing

List the names of all departments and the salaries of all employees in their department.

sql

編輯

```
SELECT department_name, name, salary
```

```
FROM staff; employees e
```

```
JOIN departments d ON e.department_id = d.id;
```

基於深度學習

缺點:

- 需要大量標註資料集, 在新SQL
- 如果模型在訓練中沒有見過符合語法結構, 但無法執行的SQL

department_name	name	salary
Engineering	Alice	50000
HR	Bob	45000
Engineering	Charlie	55000
Marketing	David	47000

employees 表格

id	1	2	3	4
name	Alice	Bob	Charlie	David
department_id	101	102	101	103
salary	50000	45000	55000	47000

departments 表格

id	101	102	103
department_name	Engineering	HR	Marketing

List the names of all departments and the salaries of all employees in their department.

sql

編輯

```
SELECT department_name, name, salary
```

```
FROM staff; employees e
```

```
JOIN departments d ON e.department_id = d.id;
```

基於預訓練語言模型

預訓練語言模型憑藉強大的語意解析能力，並透過預訓練架構與微調，成為新的主流方法，將效能提升到更高的層級

1. 預訓練架構(Pre-training Architectures)

通過在大規模語料庫上進行訓練，預訓練語言模型學習語言的結構和語義，為後續的語言理解和生成任務提供基礎能力

2. 微調(Fine-tuning)

通過微調預訓練模型(例如T5 和 GPT)，學會如何將自然語言查詢映射到具體的 SQL 查詢上

基於預訓練語言模型

優點:

- 在大規模文本資料上預訓練, 具備優秀的語義理解能力
- 支持零樣本或少樣本學習, 能夠快速應用於新場景

缺點:

- 訓練與推理成本高

基於大型語言模型

隨著模型大小和訓練數據的持續增長，預訓練語言模型自然演變為基於大型語言模型的方法，透過提示工程和微調展現出更好的能力

1. Prompt Engineering

- 設計良好的 Prompt 能清楚表達資料庫結構與使用者問題
- 利用語境理解與上下文能力，生成語法正確且語意準確的SQL

2. Fine-tuning

- 高效微調：僅調整模型少部分參數 (Parameter-Efficient Fine-Tuning)
- 完全微調：調整所有參數，讓模型全面適應特定任務 (Fully Fine-Tuning)

實作流程



Question Understanding

將使用者的自然語言轉換為對應的語義表示，確保生成的 SQL 查詢與使用者的意圖一致



Schema Comprehension

識別資料庫中的表格和欄位，確保 SQL 查詢能夠正確匹配資料庫架構



SQL Generation

根據前兩個步驟，產生正確且可執行的 SQL 查詢

```
{
  "select": ["full_name"],
  "where": {
    "job_exp": {
      "operator": ">",
      "value": 5
    }
  },
  "order_by": {
    "column": "job_exp",
    "order": "DESC"
  }
}
```

問句關鍵詞片段	資料庫欄位名稱
員工姓名	full_name
工作經驗	job_exp
資料表名稱假設為：employee_info	

「找出工作經驗超過 5 年的員工姓名，並依照工作經驗由高到低排序」

```
SELECT full_name
FROM employee_info
WHERE job_exp > 5
ORDER BY job_exp DESC;
```


目前挑戰

1. 資料隱私風險

企業在使用 ChatGPT、GPT-4 API 進行 Text-to-SQL 查詢時，需傳輸資料庫 Schema、KPI、樣本資料，無法確保敏感訊息不會洩露給API 供應商 (如 OpenAI、Google)

2. 複雜資料庫 & 評測基準不足

現實世界的資料庫結構遠比標準資料集複雜、LLM 記憶長度不足，影響 SQL 生成準確性

涉及多表 (200-1000+)、
須處理複雜查詢 (多層嵌套查詢、群組等)
例如：大型企業 ERP 系統 (如 SAP) 擁有上
萬張關聯資料表，且欄位命名常為無意義的
縮寫 (如 BKPF, BSEG)

3. 缺乏專業領域知識

LLM 訓練於開放語料庫，無法理解專業SQL 查詢的領域知識

03 資料 集

資料集分類

1. 原始資料集 (Original Datasets)

- 原生設計就是為了 Text-to-SQL 任務, 經過完整設計與標註的資料集。
- 通常包含:
 - 完整的 schema(資料庫結構)
 - 多筆問句與對應的 SQL 查詢(即一筆筆的 instance)
- 提供標準統計資訊(例: 資料庫 數量、資料表數量、平均資料列數)
- 例如: Spider、BIRD

2. 後期標註資料集 (Post-annotated Datasets)

- 在原始資料集的基礎上 **進行改編和標註的資料集**
 - 例如語言翻譯、新的標註格式等, 用來符合某些研究或應用的特殊需求
- 例如: CSpider、Spider-Vietnamese 將原始英文查詢翻譯成其它語言

特徵分類

1. 跨領域資料集 (Cross-domain Dataset)

資料集涵蓋多種**不同主題/領域**的資料庫 (如學校、醫院、公司等), 而不是單一類型。
例如: Spider

2. 知識增強資料集 (Knowledge-augmented Dataset)

加入額外的領域知識或背景資訊, 幫助模型理解**專有名詞**或進行**推理**, 更準確地生成 SQL。例如: BIRD、Spider-DK

3. 上下文依賴資料集 (Context-dependent Dataset)

適用於**多輪對話式 SQL 查詢**產生, 也就是查詢之間具有連貫語意。

模型必須理解上下文、前文查詢, 並正確處理省略或代詞 (如「他」、「上次的那個部門」)。例如: SParC、CoSQL

特徵分類

4. 穩健性測試資料集 (Robustness Dataset)

用來測試模型在「輸入擾動或語言變體」下的表現，例如：同義詞、語序變化、拼字錯誤。

評估模型是否過度依賴模板或固定格式。例如：ADVETA、Spider-SYN

5. 跨語言資料集 (Cross-lingual Dataset)

針對非英文語境，測試模型是否能處理多語查詢。例如：CSpider、DuSQL

WikiSQL

1. 資料來自Wikipedia, 雖然在程式和資料庫的數量上較大, 但它們只包含簡單的SQL查詢和單一的表格
2. 主要涉及單一的SELECT 列(包含簡單的聚合函數)與WHERE 條件, 沒有涉及到更複雜的操作, 如JOIN、GROUP BY 或 ORDER BY。
3. 由於表格結構扁平(資訊都在同一張表裡)且欄位名稱通常與自然語言問題高度重疊, 模型不需要具備真正的關聯推理能力, 只需簡單的關鍵字比對就能答對

Spider

1. 大型、複雜且跨領域的資料集, 包含了200 個資料庫, 涵蓋多個表格, 10,181 個問題和 5,693 個複雜的 SQL 查詢
 2. 訓練資料和測試資料來自不同的資料庫, 要求模型能夠對新的資料庫和查詢進行泛化
 3. 查詢不再僅僅是單表查詢, 而是包含多表JOIN、嵌套查詢等複雜的SQL結構
- 來自學術與教學資源的資料庫: 約70 個
 - 來自 DatabaseAnswers 的資料庫: 約 40 個
 - 來自 WikiSQL 的資料庫: 約 90 個

BIRD

1. 希望能更好的應對現實世界中規模較大且有噪聲的資料庫，提出了許多新的挑戰，包括資料庫中的錯誤數據、自然語言問題與資料庫值之間的外部知識對接

BIRD

需將包含特殊字符的薪水從
字串轉為浮點數

Large and Realistic Database Values



What is the **average salary** of the worst performing managers?

```
SELECT AVG(CAST(REPLACE(SUBSTR(T1.salary, 4), ',', '' ) AS REAL)) FROM  
employee AS T1 JOIN position AS T2 ON T1.positionID = T2.positionID  
WHERE T1.performance = 'Poor' AND T2.positiontitle = 'Manager'
```



Reasoned Database:

Employees

em_id	last_name	first_name	salary
0000	Milgrom	Santa	US\$57,500.00
2222	Adams	Sandy	US\$19,500.00
6543	Wood	Emily	US\$69,000.00
... ..			

BIRD

要求模型能夠推理出哪些帳戶是有資格獲得貸款的

External Knowledge Reasoning



List account id who chooses **weekly issue issuance** statement?

External Knowledge:

'POPLATEK TYDNE' stands for weekly issuance.

```
SELECT account_id FROM account WHERE account.frequency = 'POPLATEK TYDNE' ;
```



捷克語的每週發行



How many accounts are **eligible for loans** in New York City?

External Knowledge:

The condition of loans is that the type of the account should be "OWNER".

```
SELECT COUNT(*) FROM account WHERE account.type = 'OWNER' AND city = 'NY';
```



BIRD

2. 首次將 SQL 執行效率納入基準 (Valid Efficiency Score, VES), 以往的 Text-to-SQL 模型, 只評估:

- 語法正確 (是否能順利執行)
- 語意正確 (是否查到對的資料)

SQL Execution Efficiency



Among the coaches who have served more than 2 NBA teams, during which coach's period of coaching, a team has the least numbers of games lost in the post-season games?

SQL₁: normal semantic parser Run time: **22.4s**

```
SELECT coachID FROM coaches WHERE lgID='NBA' AND post_wins !=0  
AND post_losses !=0 AND coachID IN  
(SELECT coachID FROM coaches WHERE lgID='NBA' GROUP BY coachID  
HAVING COUNT(tmID)>=2) ORDER BY post_losses ASC LIMIT 1 ;
```

SQL₂: **efficient** semantic parser Run time: **4.0s**

```
SELECT coachID FROM coaches WHERE lgID='NBA' AND post_wins !=0  
AND post_losses !=0 AND EXISTS (SELECT 1 FROM coaches AS coaches1  
WHERE (coaches1.lgID='NBA') AND (coaches.coachID=coaches1.coachID)  
GROUP BY coaches1.coachID HAVING count(coaches1.tmID) >= 2  
ORDER BY NULL ) ORDER BY coaches.post_losses ASC LIMIT 1
```

BIRD

3. 除了80個開源的關聯資料庫外，還額外策劃了15 個隱藏測試集（不公開），以防止LLM 背答案作弊（LLM在訓練時幾乎看過網路上所有的公開資料）

- Kaggle:32 %
- CTU Prague Relational Learning Repository:48%
- 自建的20%

關聯資料庫 (Relational Database) 將資料存儲在具有行和列的**表格**中，這些表格稱為關聯 (relations)，並通過特定的關聯規則（不同資料表如何透過外鍵、主鍵來建立連接）來組織和管理資料。

Spider 2.0

Rank	Method	Score
1 Nov 2, 2024	Spider-Agent + o1-preview	17.01
2 Nov 2, 2024	Spider-Agent + GPT-4o	10.13
3 Nov 2, 2024	Spider-Agent + Claude-3.5-Sonnet	9.02
4 Nov 2, 2024	Spider-Agent + GPT-4	8.86
5 Nov 2, 2024	Spider-Agent + Qwen2.5-72B	6.17
6 Nov 2, 2024	Spider-Agent + DeepSeek-V2.5	5.22
7 Nov 2, 2024	Spider-Agent + Gemini-Pro-1.5	2.53
8 Nov 2, 2024	Spider-Agent + Llama-3.1-405B	2.21

來源: <https://spider2-sql.github.io/>

1. 提供了一個更加真實且複雜的測試, 挑戰當前的大型語言模型在處理企業級複雜查詢時的能力(包含超過3000個欄位的資料庫)
2. 像 GPT-4o 這樣的廣泛使用模型, 在 Spider 2.0 上的成功率僅為10.1%, 而在 Spider 1.0 上則達到86.6%, 這顯示出 Spider 2.0 的挑戰性遠超過 Spider 1.0

Spider vs Spider 2.0

Extra Hard

What is the average life expectancy in the countries where English is not the official language?

```
SELECT AVG(life_expectancy)
FROM country
WHERE name NOT IN
  (SELECT T1.name
   FROM country AS T1 JOIN
   country_language AS T2
   ON T1.code = T2.country_code
   WHERE T2.language = "English"
   AND T2.is_official = "T")
```

Spider 自然語言問題與SQL查詢

Q: I want to know the preferences of customers who purchased the Google Navy Speckled Tee in December 2020. What other product was purchased with the highest total quantity alongside this item?

```
SELECT 'Google Navy Speckled Tee' AS selected_product
),
PurchaseEvents AS (
  SELECT
    user_pseudo_id,
    items
  FROM
    `bigquery-public-
data.ga4_obfuscated_sample_ecommerce.events_*`
  WHERE
    _TABLE_SUFFIX BETWEEN '20201201' AND '20201231'
    AND event_name = 'purchase'
),
ProductABuyers AS (
  SELECT DISTINCT
    user_pseudo_id
  FROM
    Params,
    PurchaseEvents,
    UNNEST(items) AS items
  WHERE
    items.item_name = selected_product
)
SELECT
  items.item_name AS item_name,
  SUM(items.quantity) AS item_quantity
FROM
  Params,
  PurchaseEvents,
  UNNEST(items) AS items
WHERE
  user_pseudo_id IN
  (SELECT user_pseudo_id FROM ProductABuyers)
  AND items.item_name != selected_product
GROUP BY 1
ORDER BY item_quantity DESC
LIMIT 1;
```

Spider2.0 自然語言問題與SQL查詢

04

評估指標

1. 基於內容匹配的評估指標 (Content Matching-based Metrics)

- **Component Matching (CM)**

針對 SQL 查詢的不同部分 (SELECT、WHERE、GROUP BY 等), 計算其 F1 score, 來評估生成 SQL 與標準 SQL 之間的匹配程度

- **Exact Matching (EM)**

只有當所有 SQL 組件都完全匹配時才視為正確, 計算 SQL 完全匹配的比例

2. 基於執行結果的評估指標 (Execution-based Metrics)

- **Execution Accuracy (EX)**

執行預測的 SQL 語句, 並與標準 SQL 的執行結果對比, 判斷 SQL 是否正確

- **Valid Efficiency Score (VES)**

預測 SQL 語句的執行時間與標準 SQL 語句的執行時間之間的對比

05

框架介紹

DIN-SQL (2023)

Spider 排行榜 (2025/05/09)

Rank	Model	Execution Accuracy (Test)	Exact Match Accuracy (Test)	Execution Accuracy (Dev)	Exact Match Accuracy (Dev)	Extra Training Data	Paper	Code	Result	Year	Tags
1	XiYan-SQL	89.65				×	A Preview of XiYan-SQL: A Multi-Generator Ensemble Framework for Text-to-SQL	GitHub	PDF	2024	
2	PET-SQL	87.6	66.6			×	PET-SQL: A Prompt-Enhanced Two-Round Refinement of Text-to-SQL with Cross-consistency	GitHub	PDF	2024	
3	DAIL-SQL + GPT-4 + Self-Consistency	86.6		84.4	74.4	×	Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation	GitHub	PDF	2023	
4	DIN-SQL + GPT-4	85.3	60			×	DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction	GitHub	PDF	2023	

- 提供一種更輕量的解法 不需要 fine-tune, 只透過 In-Context-Learning(ICL), 可直接應用於現有的 LLM

研究問題

1. LLM 雖然強, 但在 Text-to-SQL 任務上不如 fine-tune 模型表現好, 傳統做法會對模型進行很多微調, 但 fine-tune 成本高, 彈性差。能不能只透過 **in-context learning** 就讓 LLM 表現好?
2. Text-to-SQL 其實很需要邏輯思考與步驟清楚, 所以如果把任務 **分解成小任務** 給模型, 有沒有辦法提昇推理能力?

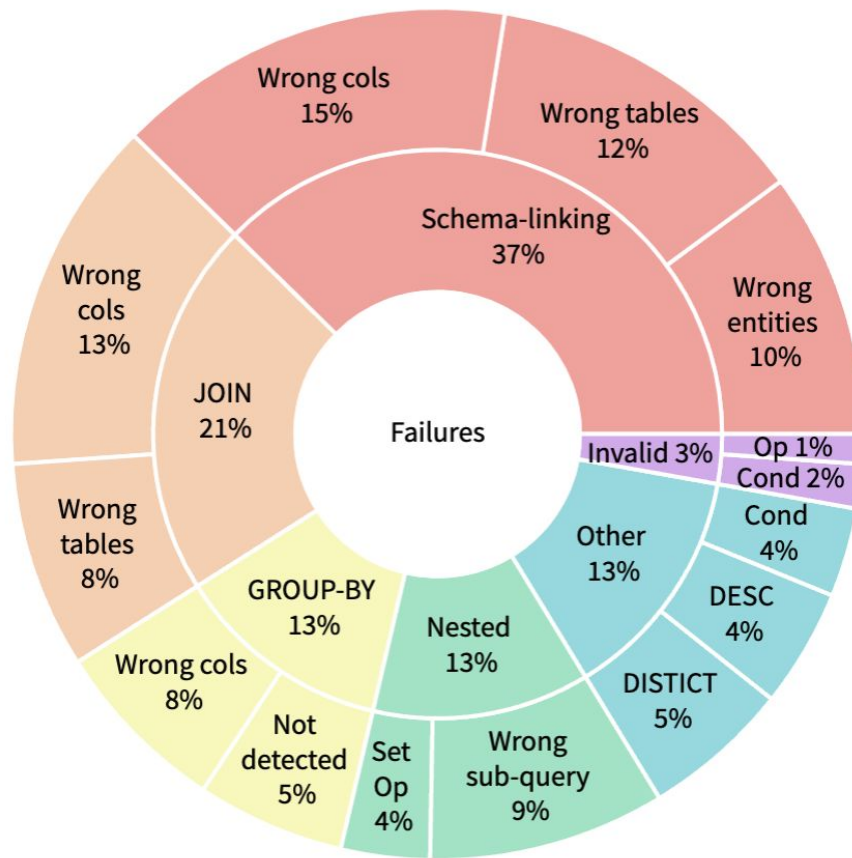


Figure 1: Statistics of simple few-shot failures using CodeX Davinci (Op refers to operators, Cond refers to conditions, and cols refers to columns)

方法

1. 將 text-to-SQL 分解成小任務 (ex: 要用到哪個資料表、哪些欄位、篩選條件) (schema linking module)
2. 然後把每個小任務丟給 LLM 處理, 最後拼湊成完整的 SQL query (query classification and SQL generation)
3. 如果某一步答案怪怪的, 再用 LLM 檢查並修正 (self-correction module)

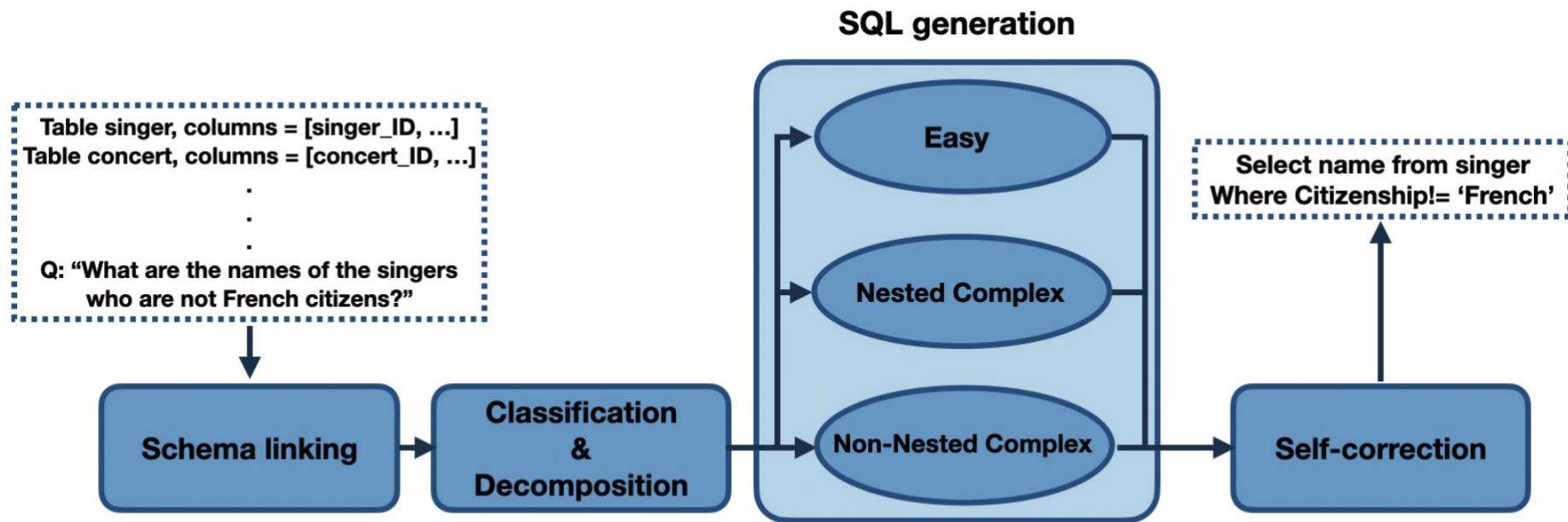
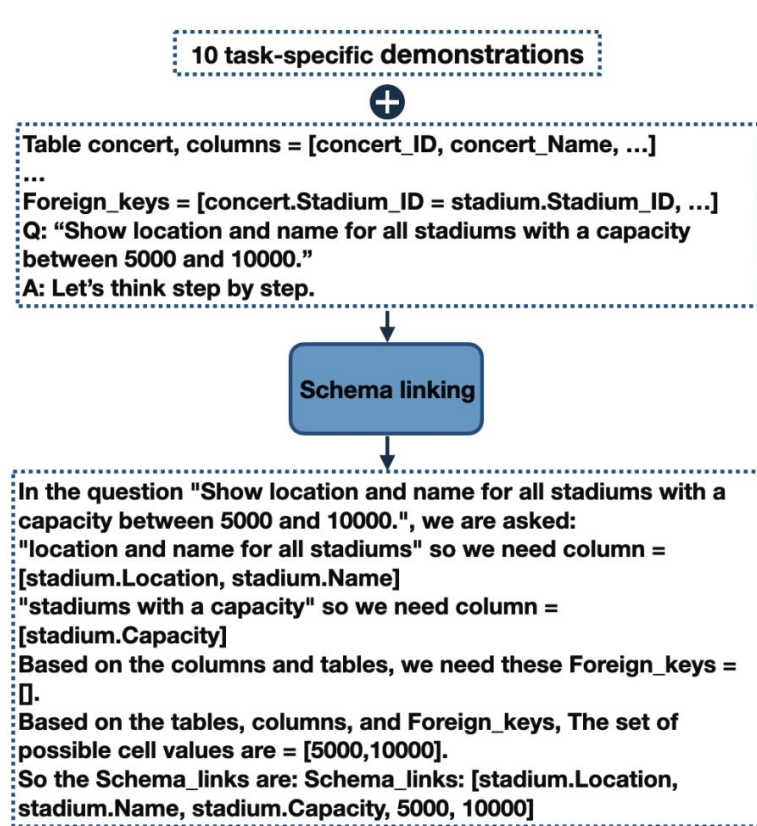
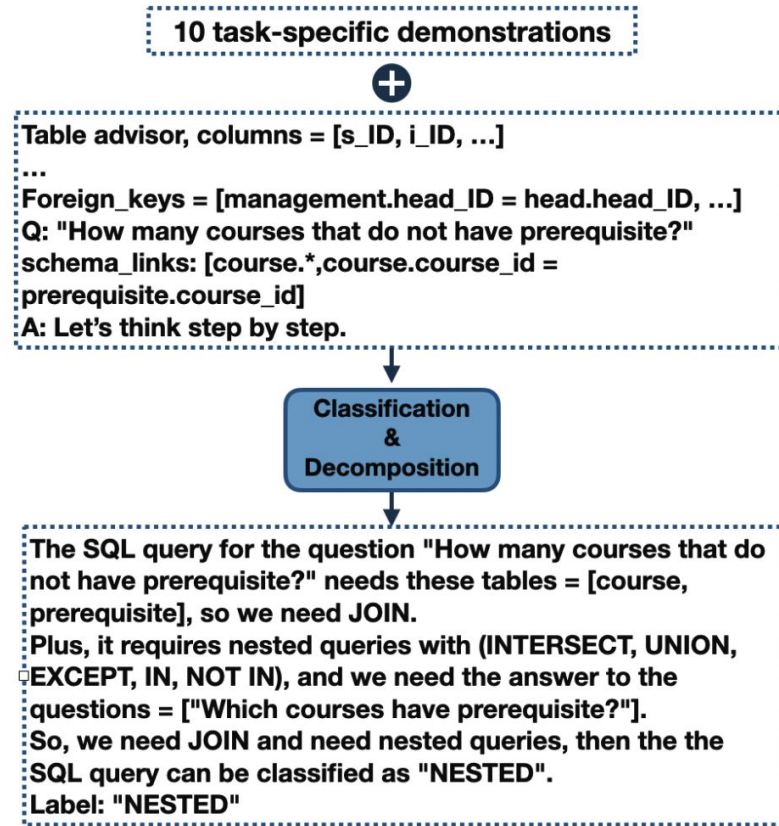


Figure 2: An overview of the proposed methodology including all four modules



(a) schema linking module



(b) classification and decomposition module

Figure 3: Examples showing the input and output of schema linking (left) and classification and decomposition (right)

結果

1. 提供一種更輕量的解法：

不需要 fine-tune, 只透過 in-context-learning, 可直接應用於現有的 LLM

2. 大幅提升 LLM 在 Text-to-SQL 任務中的準確率：

在 Spider 資料集上的準確率達到 85.3%, 超過 原本最好的 79.9% (2023/04)

遇到的挑戰

1. 需要大量的範例：

每一種查詢類別都需要對應的範例

2. 處理成本高：

以 GPT-4 為例，每處理一個 Spider 資料集的自然語言查詢，大約需要 0.5 美元、60 秒處理時間（呼叫次數多&非平行化的處理步驟）

3. 可擴展性限制：

成本與速度問題限制了該方法在大型應用場合的可能性

XiYan-SQL (2024)

Spider 排行榜 (2025/05/09)

Rank	Model	Execution Accuracy (Test)	Exact Match Accuracy (Test)	Execution Accuracy (Dev)	Exact Match Accuracy (Dev)	Extra Training Data	Paper	Code	Result	Year	Tags
1	XiYan-SQL	89.65				×	A Preview of XiYan-SQL: A Multi-Generator Ensemble Framework for Text-to-SQL			2024	

1. 由阿里巴巴集團提出的多生成器集成Text-to-SQL 框架
2. 曾在 Spider 及 Bird 資料集中排名第一

研究問題

1. LLM 常常不知道資料庫欄位之間的關係, 或是欄位名稱所代表的意義
>> 提出 M-Schema 資料庫結構表示方式
2. ICL 可以生成多種 SQL 但不一定精準, SFT 雖然比較可控但面對比較複雜的問題容易產生錯誤
>> 用多種模型生成再整合結果
3. 在透過 ICL 做 text-to-sql 時, 模型容易受到 entity 的影響
>> 提出 Skeleton Similarity 策略來做 ICL
4. LLM 雖強但缺乏可控性 (最終決策)
>> 使用 fine-tuned 的模型來選出最符合語意的 SQL 碼作為最終結果

研究問題 1

LLM 常常不知道資料庫欄位之間的關係，或是欄位名稱所代表的意義

>> 提出 M-Schema 資料庫結構表示方式如下圖

資料庫名稱

`【DB_ID】 world_population`

該資料庫包含之資料
表 & 欄位資訊

```
# Table: city_population
[
  ( city_id: INT, Primary Key, Examples: [1,2,3] ),
  ( country_id: INT, Maps to country_info.country_id, Examples: [100,101] ),
  ( city_name: TEXT, Examples: ["Taipei", "Kaohsiung"] ),
  ( year: INT, Examples: [2015, 2016, 2019] ),
  ( population: INT, Examples: [2000000, 1500000] )
]

# Table: country_info
[
  ( country_id: INT, Primary Key, Examples: [100,101] ),
  ( country_name: TEXT, Examples: ["Taiwan", "Japan"] )
]
```

外鍵關聯

`【Foreign Keys】`

`city_population.country_id = country_info.country_id`

研究問題 1

M-Schema 根據 [MAC-SQL Schema](#) 加以改進, M-Schema 相比 MAC-SQL Schema 多了更豐富的欄位資訊, 輸入給 LLM 讓 LLM 更理解資料表的結構關係

DDL Schema	MAC-SQL Schema	M-Schema
<pre>CREATE TABLE hero_power (hero_id INTEGER, power_id INTEGER, FOREIGN KEY (power_id) REFERENCES superpower(id)); CREATE TABLE superpower (id INTEGER PRIMARY KEY, power_name TEXT);</pre>	<pre>【DB_ID】 superhero 【Schema】 # Table: hero_power [(hero_id, hero id.), (power_id, power id.)] # Table: superpower [(id, id.), (power_name, power name, Value examples: ['Agility'].)] 【Foreign keys】 hero_power.hero_id = superpower.id</pre>	<pre>【DB_ID】 superhero 【Schema】 # Table: hero_power [(hero_id INTEGER, Primary Key, the id of the hero Maps to superhero(id), Examples: [1, 2, 3]], (power_id INTEGER, the id of the power Maps to superpower(id), Examples: [1, 18, 26])] # Table: superpower [(id INTEGER, Primary Key, the unique identifier of the superpower, Examples: [1, 2, 3]), (power_name TEXT, the superpower name, Examples: [Agility, Accelerated Healing, Lantern Power Ring])] 【Foreign keys】 hero_power.power_id=superpower.id</pre>

研究問題 2

ICL 可以生成多種 SQL 但不一定精準,
SFT 雖然比較可控但面對比較複雜的問題容易 產生錯誤

*In-Context Learning (ICL) :

不微調模型，僅透過在 Prompt 中提供範例來讓模型學會目標任務，這種方式彈性大，可以快速應用到不同任務，但是輸出結果比較不可控，每次產生的結果可能不同

*Supervised Fine-Tuning (SFT) :

透過標註過的資料對模型進行微調這種方式結果較穩定且可控但當任務太複雜或資料涵蓋不夠全面時模型表現可能會下降

研究問題 2

XiYan-SQL 分別**使用 ICL 與 SFT 產生多個 SQL 生成器**，這些生成器基於不同風格的訓練資料與不同的 Prompt 進行訓練。

每個生成器產生候選 SQL 後，系統再從中選出最好的結果，從而結合兩種方法的優點

ICL SQL Generator

透過 **Skeleton Similarity** 做 ICL

原問題: "What's the population of China in 2020?"

骨架處理後變成: "What's the population of <country> in <year>?"

再根據此骨架，選出對應範例作為 Prompt 輸入

Fine-tuned SQL Generator

fine-tune有兩階段:

1. 讓模型學會基本的 text-to-sql 能力
2. 強化模型產生不同風格 SQL 的能力
(ex: 同一個句子可以用巢狀或是JOIN寫)

研究問題 3

在透過 ICL 做 text-to-SQL 時，模型容易受到 entity 的影響

-- 範例 1

Q: List the names of authors who were born in China.

A: SELECT name FROM authors WHERE country = 'China';

-- 使用者問題

Q: List the names of authors who were born in America.

模型輸出: SELECT name FROM authors WHERE country = 'China';

在以上範例可以發現，模型機械性的複製了'**China**' 這個 entity 作為模型的輸出，因為 LLM 在 ICL 中傾向使用最顯眼的詞彙，所以如果特定 entity 在範例中重複出現，可能會讓模型過度強調，反而忽略掉正確的查詢結構

研究問題 3

XiYan-SQL 提出了 Skeleton-similarity 去除範例中的 entity 來做 ICL

1. 透過 NLTK 這類工具辨識自然語言中的entities, 例如國家、地名、人物等
2. 把相同類別的entity 換成統一的標記 例:

「China」和「America」取代成 <country>

3. 把問題骨架化 例

原問題: "What's the population of China in 2020?"

骨架處理後變成: "What's the population of <country> in <year>?"

4. 把骨架化的句子轉成向量, 計算與範例集中其他範例的相似度, 選出Top-K 相似的範例做 ICL

研究問題 4

LLM 雖強但缺乏可控性

雖然 GPT-4、Gemini 等封閉模型可以靠 ICL 得到良好的結果，但無法保證每次生成的 SQL 是一致且正確的，一般來說會使用 **Self-Consistency** 來解決每次生成結果不一致的問題，但這個方法的計算成本太高，而且有可能最常見的結果是錯誤的

***Self-Consistency :**

生成多個候選結果後(通常為10到20個)，統計出最常見的來當作最終結果

研究問題 4

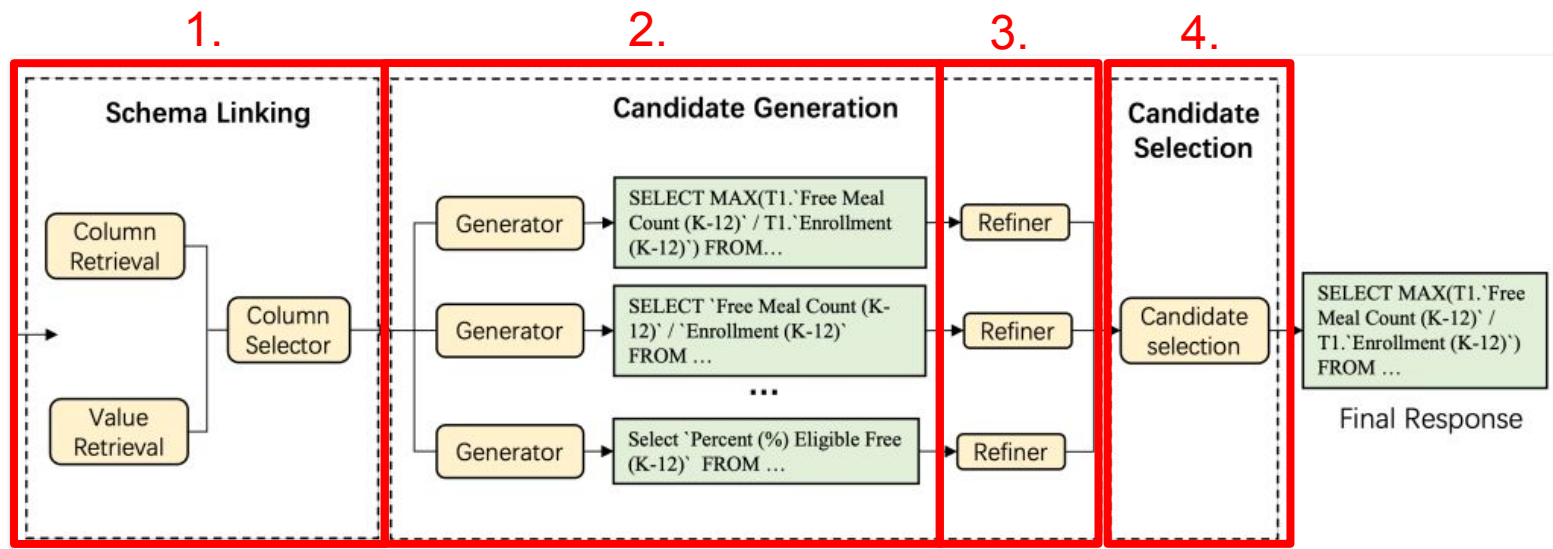
不使用傳統的 **Self-Consistency**, XiYan-SQL 特別 fine-tune 了一個模型來選擇多個候選 SQL:

把候選 SQL 全部執行一次, 先透過 **Refiner** 判斷這些 SQL 執行結果是否正確, 最後透過 fine-tune 過的模型選出最符合原句語意的一個

***Refiner :**

Refiner 為 XiYan-SQL 提出的修正模組, 每當生成器生成一條候選SQL, Refiner 就會去執行 SQL 結果, 如果有錯誤則將結果輸入給LLM 做修正

整體流程



1. Retrieval Module 從使用者輸入中取出 column 及 entity 並確定對應哪些 column, 最後整理成 M-Schema
2. 將使用者輸入的語句與轉換成的 M-Schema 輸入給多個 Generator 產生候選 SQL
3. 每個 Generator 產生的 SQL 都會經過 Refiner 檢查錯誤
4. 使用 fine-tined 的模型來選出最符合語意的 SQL 碼作為最終結果

遇到的挑戰

1. 若資料庫裡有非常多的表及欄位，可能無法在**有限的時間**過濾並選出最相關的欄位
2. XiYan-SQL 需要多個 Generator 各生成一個或數個候選 SQL，再加上修正器、選擇模型，整個推理過程中需要多次 LLM 調用，消耗**成本巨大**
3. Refiner 會依據執行結果來做修正，但如果出錯的原因很隱諱**語意錯誤**但語法沒錯
refiner 可能就無法處理這類問題
 - a. Query: 查詢 2020 年所有來自中國的申請者的平均年齡
 - b. Generated SQL: **SELECT AVG(age) FROM applicants WHERE nationality = 'China'** ; 忘了加上 AND year = 2020
 - c. 語法正確、可以執行，而且也會回傳一個平均值，但那不是使用者真正要的答案